

TEKNOFEST

**HAVACILIK, UZAY VE TEKNOLOJİ
FESTİVALİ**

**ULAŞIMDA YAPAY ZEKA YARIŞMASI
KRİTİK TASARIM RAPORU**

TAKIM ADI

Glieser

BAŞVURU ID

468514

İÇİNDEKİLER

ŞEKİL LİSTESİ	3
1. TAKIM ŞEMASI (5 PUAN).....	5
2. PROJE MEVCUT DURUM DEĞERLENDİRMESİ (15 PUAN)	6
3. ALGORİTMALAR VE SİSTEM MİMARİSİ (25 PUAN)	7
3.1. Veri Setleri (10 Puan).....	7
3.2. Algoritmalar (15 Puan)	8
3.2.1. YOLOv4.....	8
3.2.2. OBJECT DETECTION	9
3.2.3. CMake	9
3.2.4. OpenCV.....	9
4. ÖZGÜNLÜK (25 PUAN)	10
5. SONUÇLAR VE İNCELEME (25 PUAN).....	11
6. KAYNAKÇA	12



ŞEKİL LİSTESİ

Şekil 1	7
Şekil 2 Nesne alanı karşılaştırması	7
Şekil 3 Open Image Dataset'te görsellerin sınıflandırması	7
Şekil 4 Sınırlayıcı Kutunun Çalışma Mantığı	8
Şekil 5 IOU çalışma mantığı	9
Şekil 6 YOLO sınırlayıcı kutu oluşturma adımları	9
Şekil 7	11
Şekil 8	11



TABLO LİSTESİ

Tablo 1 Takım Şeması	5
Tablo 2.....	11



1. TAKIM ŐEMASI (5 PUAN)

Tablo 1 Takım Őeması

Ulař Alptekin (Kaptan)	-Algoritmaların arařtırılması. -Yazılım mimarisinin arařtırılması, programlanması. -Raporun oluřturulması.
Emre Onur Őetin	-Yazılım mimarisinin arařtırılması, programlanması. -Eđitim sũrecinin optimize edilmesi.
Ceylin Ceylan	-Proje takviminin hazırlanması. -E-posta grubunu, resmi web sitelerini ve sosyal medya hesaplarını takip edilmesi. -Raporun oluřturulması.
Anıl Gũkhan Ayhan	-Konu hakkında bilimsel arařtırma yapılması. -Verilerin toplanması, sınıflandırılması. -Raporun oluřturulması
Berk Jiyan Yıldız (iletiřim sorumlusu)	-Algoritmaların arařtırılması. -Yazılım mimarisinin arařtırılması, programlanması. -Raporun oluřturulması.



2. PROJE MEVCUT DURUM DEĞERLENDİRMESİ (15 PUAN)

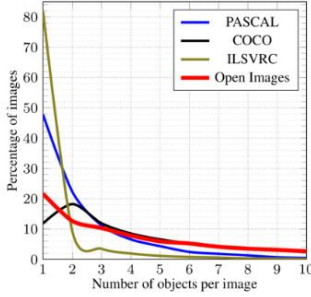
Ön tasarım raporunda yaptığımız değişiklik YOLOv5 yerine YOLOv4'ü kullanmak oldu. Nesne tespiti şu anda istediğimiz gibi çalışmakta. YOLOv4, nesne algılamalarını gerçekleştirmek için derin erişimli sinir ağlarını kullanan son teknoloji bir algoritmadır, son derece hassastır ve YOLOv4'ün çıkması ile tek bir GPU üzerinden görüntü alınabilir hale geldi. [1] Darknet'in ve YOLO'nun buildini almak için visual studio 2019 kullandık. Compile etmek için Cmake yazılımını kullandık ve OpenCV, Nvidia, CUDA, görüntü işlemeyi GPU üzerinden yapmamızı sağladı. Yapay zeka eğitimi için ise Darknet yazılımını kullandık.



3. ALGORİTMALAR VE SİSTEM MİMARİSİ (25 PUAN)

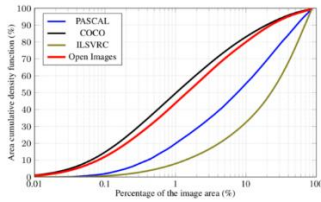
3.1. Veri Setleri (10 Puan)

Projenin veri seti ihtiyacını karşılamakta Google destekli Open Images Dataset v6 kullanıldı. Open Images Dataset, yaklaşık 9 milyon görsel, 600 nesne sınıfı ve 14.6 milyon sınırlayıcı kutu ile var olan en büyük veri setidir. Nesneleri tanımlayan sınırlayıcı kutular, doğruluk ve tutarlılığı sağlamak için profesyonel yorumcular tarafından büyük ölçüde elle çizilmiştir. Görüntüler çok çeşitlidir ve genellikle birkaç nesne içeren karmaşık sahneler içerir.



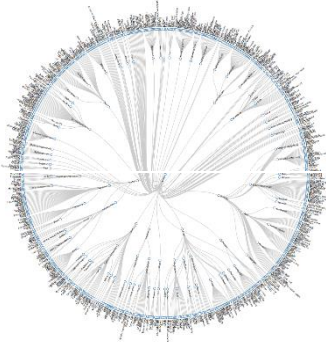
Şekil 1

Open Images Dataset fotoğraf başına öbür alternatiflerine kıyasla daha çok obje tanımlamaktadır. Bu sayede veri tasarrufu sağlanır.



Şekil 2 Nesne alanı karşılaştırması

Open Images Database'de sınıflar Freebase veya Google Knowledge Graph API'da bulunabilen MIDs(Machine-generated Ids) tarafından oluşturulmuştur. Her bir sınıfın kısa bir açıklaması Open Image Datasetin sitesinde mevcuttur.



Şekil 3 Open Image Dataset'te görsellerin sınıflandırılması

Yarışma sürecinde ihtiyacımız olan bütün veri setleri Open Images Database'de fazlasıyla mevcuttur.

3.2. Algoritmalar (15 Puan)

3.2.1. YOLOv4

YOLO, 'You Only Look Once' teriminin kısaltmasıdır. YOLO, bir resimdeki çeşitli nesneleri (gerçek zamanlı olarak) algılayan ve tanıyan bir algoritmadır. YOLO algoritması, nesneleri gerçek zamanlı olarak algılamak için evrişimli sinir ağlarını (CNN) kullanır. Algoritma nesneleri algılamak için bir sinir ağı üzerinden yalnızca tek bir ileri yayılım gerektirir.

YOLO algoritması aşağıdaki nedenlerden dolayı önemlidir:

- **Hız:** Algoritma hızlı çalışır çünkü nesneleri gerçek zamanlı olarak tahmin eder.
- **Doğruluk Oranı:** YOLO, minimum arka plan hatasıyla doğru sonuçlar sağlayan tahmine dayalı bir tekniktir.
- **Öğrenme Yetenekleri:** Algoritma, nesnelerin temsillerini öğrenmesini ve bunları nesne algılamada uygulamasını sağlayan güçlü bir öğrenme yeteneğine sahiptir.

YOLO'nun çalışma prensibi 3 tekniğe dayanır:

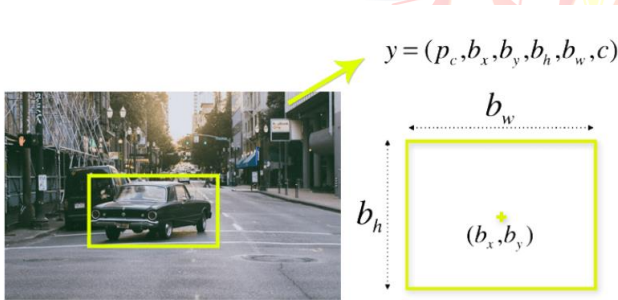
- Residual Blocks(Kalıcı Bloklar)
- Sınırlayıcı Kutu Regresyonu
- IOU (Intersection Over Union)

Residual Blocks: Görüntü çeşitli ızgaralara bölünür. Her ızgara $S \times S$ boyutuna sahiptir. Her bir ızgara içerlerinde bulunan nesneleri tanımlamakla yükümlüdür.

Sınırlayıcı kutu regresyonu: Sınırlayıcı kutular görseldeki nesnelerin etrafını saran kutulardır. Her bir kutu şu özelliklere sahiptir:

- En
- Boy
- Sınıf (kedi, insan, araba vb.)
- Kutu Merkezi

YOLO en, boy, merkez ve nesne sınıfını belirtmek için tek bir sınırlayıcı kutu kullanır.

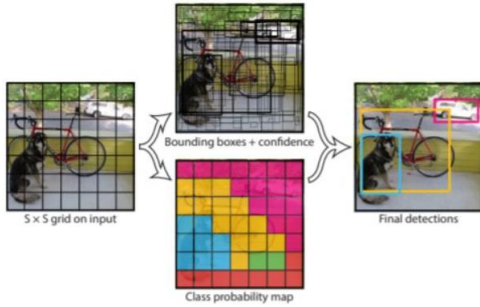


Şekil 4 Sınırlayıcı Kutunun Çalışma Mantığı

Intersection Over Union: Intersection Over Union (IOU), nesne algılamada kutuların nasıl üst üste bindiğini açıklayan bir olgudur. YOLO, nesneleri mükemmel bir şekilde çevreleyen bir çıktı kutusu sağlamak için IOU'yu kullanır. Her ızgara hücresi, sınırlayıcı kutuları ve bunların güven puanlarını tahmin etmekten sorumludur. Öngörülen sınırlayıcı kutu gerçek kutuyla aynıysa, IOU 1'e eşittir. Bu mekanizma, gerçek kutuya eşit olmayan sınırlayıcı kutuları ortadan kaldırır.



Şekil 5 IOU çalışma mantığı



Şekil 6 YOLO sınırlayıcı kutu oluşturma adımları

3.2.2. OBJECT DETECTION

Nesne tespiti, dijital görüntülerde ve videolarda belirli bir sınıftaki (insanlar, binalar veya arabalar gibi) anlamsal nesnelerin örneklerini algılamakla ilgilenen, bilgisayarla görme ve görüntü işleme ile ilgili bir bilgisayar teknolojisidir. Nesne tespiti, bilgisayarla görme ve görüntü işlemeden farklı olarak algılanan nesnenin görüntü üzerinde koordinatlarının bulunmasını içerir. Bulunan koordinatlar ile nesnenin bir çerçeve ile içine alınacağı alan da tespit edilmiş olur. Nesne tespiti en çok görüntü tanımlama, araç sayma, aktivite tanıma, yüz algılama, yüz tanıma, video nesnesi ortak segmentasyonu gibi bilgisayarla görme görevlerinde kullanılmaktadır. Ayrıca, örneğin bir futbol maçı sırasında bir topun izlenmesi, bir kriket sopasının hareketlerinin izlenmesi veya bir videodaki bir kişinin izlenmesi gibi nesnelerin izlenmesinde de kullanılır. Her nesne sınıfının, sınıfın sınıflandırılmasına yardımcı olan kendine özel özellikleri vardır. Örneğin tüm daireler yuvaraktır. Nesne sınıfı algılama, bu özel özellikleri kullanır. Örneğin, daire ararken, bir noktadan (yani merkezden) belirli bir uzaklıkta bulunan nesneler aranır. Benzer şekilde, kare ararken, köşeleri dik ve kenar uzunlukları eşit olan nesnelere ihtiyaç vardır. Göz, burun ve dudakların bulunabileceği ve ten rengi ve gözler arasındaki mesafe gibi özelliklerin bulunabileceği yüz tanımlama için de benzer bir yaklaşım kullanılmaktadır.

3.2.3. CMake

CMake, C veya C++ ile geliştirilen uygulamalar için yazılım derleme otomasyonudur. Herhangi bir programlama dili ile yazılan kodun çalıştırılabilir olması için derlenmesi gerekir. Derleme işlemi sırasında derleyiciye uygun parametrelerin eklenmesi, gerekli kütüphanelerin dahil edilmesi, birim t estlerin çalıştırılması gibi işlemler yapılır. Tüm bu işlemler için IDE kullanılabilir ancak IDE ayarları birbirinden farklıdır. C ve C++ bu işlemler için Make adından bir araca sahiptir. Ancak bu araç sadece Linux/Unix tabanlı işletim sistemlerinde çalışır. CMake temel olarak bu işleri platform ve IDE bağımsız bir şekilde yapmak için kullanılan bir araçtır. Bu araç ile oluşturulan kurallar kod üreteçleri ile farklı IDE'lerde geliştirme yapmaya imkan verir.

3.2.4. OpenCV

OpenCV (Open Source Computer Vision Library, anlamı Açık Kaynak Bilgisayar Görüşü Kütüphanesi) gerçek-zamanlı bilgisayar görüşü uygulamalarında kullanılan açık kaynaklı kütüphane. Bu kütüphane çoklu platform ve BSD lisansı altında açık kaynaklı bir yazılımdır.

4. ÖZGÜNLÜK (25 PUAN)

Takım olarak literatürde belirtmiş olduğu yazılım programlarının kullandık. Örneğin küçük nesneleri bulmakta zorluk çeken object detection modellerinin yerine YOLOv4 içerisinde bulunan farklı modülleri kullandık. YOLOv4, nesne algılamalarını gerçekleştirmek için derin erişimli sinir ağlarını kullanan son teknoloji bir algoritmadır, son derece hassastır ve YOLOv4'ün çıkması ile tek bir GPU üzerinden görüntü alınabilir hale geldi. R-CNN, Fast R-CNN ve Faster R-CNN gibi iki aşamalı (two-state) nesne takibi yapan algoritmalar da vardır. Bu sebepten ötürü YOLOv4'ü kullanmaya karar verdik. Ayrıca diğer sürümlerine göre örneğin YOLOv5'teki gibi backbone ve PA-NET neck kullanılır. Fakat YOLOv5 den farklı olarak bir PyThoch implementasyonu değildir. Sonrasında Darknet'in içerisinde bulunan Tiny-Darknet'i kullanmaya karar verdik. SqueezeNet gibi sadece parametreler için optimize edilen yazılımlar farklı boyutlardaki nesneleri tanımlayamıyor, ayrıca daha yavaş çalışıyorlardı. Darknet ve YOLOv4 içerisinden aldığımız kodların takımımızın amaçları ve hedefleri doğrultusunda çalışmasını istediğimizden ötürü bir IDE programı olan visual studio referansı ile build işlemimizi gerçekleştirdik. Bununla birlikte dosyaların düzenlenmesinde de visual studio'dan yararlandık. Toparladığımız yazılımların bir arada bulunabilmesi ve farklı programlar içerisinde kullanılan kodların eş zamanlı çalışmasını sağlamak amacıyla Cmake yazılımını kullandık. Yazılımın yenilikçi ve gelişmiş yönlü compile işlemini Cmake üzerinden gerçekleştirdik. Veri setinin görüntüleri algılayabilmesi ve tanımlayabilmesi amacıyla OpenCV kullandık. OpenCV'nin yüz tanıma özelliği; yaralanan, herhangi bir şekilde acil müdahale edilmesi gereken kişi veya kişilerin yüz tanıma özelliği kullanılarak anında tespit ve tanımlama yapılabiliyor. Sadece nesne tespiti için programın kullanılmasını önleyerek aynı zamanda yapay zeka öğrenimi için ayrı bir yenilik olmuştur. Nvidia CUDA'nın görüntüyü GPU üzerinden almamızı sağlaması ise daha stabil ve daha hızlı veri almamızı sağladı. Takımımızın en çok önemseydiği yenilikçi çözüm, yarışma tarafından belirlenen şartlara uymakla birlikte nesne tespitin en ince ayrıntısına kadar yapılması ve tespit edilen nesnelerin hızlı bir şekilde verilerinin alınmasıdır. Kullandığımız programlar ve elimizde olan materyalleri bu hedef doğrultusunda kullanmış bulunmaktayız.



5. SONUÇLAR VE İNCELEME (25 PUAN)

Tablo 1

Veri Seti	Pretrained Model	Eğitim Süresi
Komite tarafından gönderilen video sahneleri	YOLO v4	4 saat – Nvidia Geforce Gtx 1080
Komite tarafından gönderilen video sahneleri	Faster R-CNN (Resnet 50)	12 saat – Nvidia Geforce Gtx 1080

Modelin çalışmasını izlemek için ön tasarım raporunda bahsedilen YOLOv4, Open Images Dataset v6 ile de tespit nesnelerin etrafında sınırlayıcı kutuların olduğu görülmüştür. Open Images Dataset v6'nın çoklu nesne tespit özelliği sayesinde Nvidia CUDA görüntü işlemcisi GPU tarafından görüntülenen nesneleri tanımlayabildiği gözlenmiştir. İnternet bağlantısı olmadan çalışabilen bu programlar sistemimizin Wifi bağlantısına ihtiyaç duymadan, bağımsız bir şekilde aktif oluyor. Tanımlanan nesnelerin Open Images Database'in içerisinde bulunan MIDs (Machine-generated Ids) aracılığıyla nesnelerin tanımlandıktan sonra sınıflarına ayrılma işleminin tamamlandığını gördük. Fakat kullanılan ekran kartı sebebiyle Nvidia dışında diğer ekran kartları ile işlemin gerçekleştirilemediği belirlenmiştir. Gpu üzerinden görüntü işlemek için Nvidia markasına ait ekran kartları kullanılmak zorundadır. Aynı zamanda kullanılan ekran kartının gelişmiş olmasına dikkat edilmelidir. Kullandığımız veri seti olan non-maksimumun sınıf çeşitliliğinin geniş olmasından dolayı nesne tanımlanması veya sınıflandırılmasında bir sorun yaşanmamıştır. Kullanılan programların ücretsiz ve erişime açık olmalarından dolayı takımımız ihtiyacı olan materyallere ve verilere ulaşmakta güçlük çekmemiştir. Yarışma gününe kadar sistem denemeye tabi tutulacaktır.



Şekil 8



Şekil 7

6. KAYNAKÇA

- [1] D. Kasparov, «Why Chess?,» *Journal of Chess*, pp. 215-269, 2012.
- [2] V. Anderson, *Aerodynamics*, Ottawa: Helenova Publishing, 1999.
- [3] «<https://tr.wikipedia.org>,» *wikipedi*, 10 Mayıs 2022. [Çevrimiçi]. Available: https://tr.wikipedia.org/wiki/Nesne_tespiti. [Erişildi: 14 Haziran 2022].
- [4] «<https://deryacakmak.medium.com>,» 4 Kasım 2020. [Çevrimiçi]. Available: <https://deryacakmak.medium.com/yolov4-ve-yolov5-9c4ad208579e>. [Erişildi: 15 Haziran 2020].
- [5] «<https://pjreddie.com>,» *Survival Strategies for the Robot Rebellion*, 14 Haziran 2022. [Çevrimiçi]. Available: <https://pjreddie.com/darknet/tiny-darknet/>.
- [6] «<https://www.yusufsezer.com.tr>,» YusufSezer, [Çevrimiçi]. Available: <https://www.yusufsezer.com.tr/cmake/>. [Erişildi: 13 Haziran 2022].

